RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

MOBILE APPLICATION DEVELOPMENT
MCA221IA
SEE PROJECT BASED LABORATORY

College Forum App

submitted by

Priya N	**1RV25MC075**
Spandana N	**1RV25MC096**
Sumanth Bhaskar Hegde	**1RV25MC103**
Sumukha Rao B S	**1RV25MC104**

under the guidance of

Prof. Chandrani Chakravorty & Prof. Prashanth K
Assistant Professor
Department of Master of Computer Applications
RV College of Engineering

Department of Master of Computer Applications
RV College of Engineering, Bengaluru – 560059
2025-2026

RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

CERTIFICATE

Certified that the project entitled **"College Forum App"** on **Mobile Application Development – MCA221IA** has been carried out by **Priya N (1RV25MC075), Spandana N (1RV25MC096), Sumanth Bhaskar Hegde (1RV25MC103) and Sumukha Rao B S (1RV25MC104)** who have successfully completed the project for the final SEE Lab Examination, incorporating all concepts of the course conducted by the Department of MCA, RV College of Engineering, Bengaluru.

Internal Guide
Prof. Chandrani Chakravorty
Assistant Professor
Department of MCA
RV College of Engineering

Head of the Department
Dr. Jasmine K S
Associate Professor & Director
Department of MCA,
RV College of Engineering

External Viva Examination

Name of Examiners **Signature with Date**

1.

2.

Table of Contents

List of Tables

List of Figures

1. Introduction

College Forum App is a Progressive Web Application (PWA) developed to provide a centralized digital notice board for educational institutions. The platform enables administrators, faculty members, departments, clubs, and student organizations to share announcements, academic updates, events, workshops, placement opportunities, and other important information through a single communication channel. Within its interface the application is branded as “RVCE Connect”, the official notice board of RV College of Engineering.

The system implements role-based access control, where administrators manage users and departments, publishers create and distribute notices, and viewers access announcements through a personalized feed. It also supports department-specific content targeting, allowing users to receive information relevant to their academic department while still accessing institution-wide announcements.

To improve user engagement, College Forum provides features such as post likes and publisher subscriptions. As a Progressive Web Application, it offers a responsive, mobile-friendly interface, installation support, and offline accessibility through Service Workers and caching mechanisms. The application is built using HTML5, CSS3, Bootstrap 5, JavaScript, Node.js, Express.js, MySQL, and JWT-based authentication, ensuring a secure, scalable, and efficient communication platform. It also delivers real-time browser push notifications and lets every department and club operate as a subscribable community.

1.2. Overview of Problem Domain

Educational institutions generate a large volume of information, including examination schedules, placement drives, workshops, seminars, events, and administrative announcements. Ensuring that this information reaches the right audience at the right time is a significant challenge.

Traditional communication methods such as physical notice boards, emails, and messaging groups often result in delayed communication, information overload, and missed announcements. Additionally, the lack of targeted communication makes it difficult for students to access notices that are relevant to their departments or interests.

The problem domain focuses on developing a centralized and efficient communication system that supports secure information sharing, role-based access, department-specific announcements, and easy accessibility across devices. College Forum addresses these challenges by streamlining communication and providing a unified platform for campus-wide information management.

2. Key Features and Analysis

College Forum is designed as a centralized digital communication platform for educational institutions. The system combines secure role-based access, targeted information delivery, interactive engagement features, and Progressive Web App capabilities to provide a modern and efficient campus communication solution. The following are the major features of the system along with their analysis. Figure 1 summarizes these key features and shows how they interact within the platform.

2.1 Role-Based Access Control

The system supports three distinct user roles: Admin, Publisher, and Viewer.

- Admin: Manages users, departments, and overall system operations.
- Publisher: Faculty members, departments, or club representatives who can create and publish notices.
- Viewer: Students who can view notices, like posts, and subscribe to publishers.

This role-based structure ensures that only authorized users can publish content, maintaining the authenticity and reliability of announcements. It also prevents misuse of the platform and provides a clear separation of responsibilities within the system.

2.2 Department-Specific Notice Distribution

One of the core features of College Forum is the ability to target announcements to specific departments or groups. Publishers can choose whether a notice should be visible to:

- Everyone in the institution
- A single department
- Multiple selected departments

This feature reduces information overload by ensuring that students receive only the notices relevant to them. For example, a workshop organized by the MCA department can be shown only to MCA students, while institution-wide announcements remain visible to all users.

2.3 Publisher Subscription System

Students can subscribe to publishers such as departments or clubs. Once subscribed, users can easily track updates from those publishers in their personalized feed. This feature improves user engagement and allows students to stay connected with organizations and activities that match their interests.

2.4 Post Interaction and Engagement

The platform allows viewers to like posts, with each user having only one like per post. This interaction mechanism helps measure the reach and engagement of announcements. Publishers

and administrators can understand which notices are receiving more attention, making communication more effective and data-driven.

2.5 Progressive Web App (PWA) Support

College Forum is implemented as a Progressive Web Application, providing an app-like experience directly through the browser. Users can install the application on their devices without downloading it from an app store. The PWA approach offers: [4]

- Fast loading performance
- Responsive design for mobile and desktop devices
- Home screen installation
- Improved user experience similar to native applications
- Installable on Android, iOS, and desktop directly from the browser
- Launches in a standalone, full-screen window like a native application

This makes the application highly accessible and convenient for students and faculty members.

2.6 Offline Access Through Service Workers

The application uses Service Workers and caching mechanisms to store the app shell and recent posts locally on the device. As a result, users can access previously loaded notices even when internet connectivity is unavailable. This feature is especially useful in areas with unstable network connections and ensures continuous access to important information.

- Caches the application shell (HTML, CSS, and JavaScript) for instant loading
- Stores recently viewed notices locally so they remain readable offline
- Serves a dedicated offline fallback page when no cached content is available
- Automatically refreshes the cache once network connectivity is restored

2.7 Secure Authentication with JWT

College Forum uses JSON Web Token (JWT) authentication to secure user sessions and API access. After successful login, users receive a token that is used to authenticate subsequent requests. This approach provides:

- Secure session management
- Protection of restricted APIs
- Scalable and stateless authentication

The system also enforces authorization checks based on user roles, ensuring that users can access only the features permitted to them.

2.8 Mobile-First Responsive Interface

The frontend is built using Bootstrap 5 with a mobile-first design approach. The interface adapts seamlessly to smartphones, tablets, laptops, and desktop screens. Since students primarily access information through mobile devices, this design ensures better usability and accessibility across all platforms.

- Mobile-first layout built on Bootstrap 5's responsive grid system
- Adapts fluidly to smartphones, tablets, laptops, and desktop screens
- Bottom navigation bar optimised for one-handed mobile use
- Touch-friendly controls and readable typography on small screens

2.9 Real-Time Push Notifications

Beyond the in-app feed, College Forum App supports genuine browser push notifications using the Web Push protocol with VAPID keys. For every community a viewer can switch on a per-community bell; when a publisher of that community posts a new notice, the server fans out a push message so that subscribers are alerted even when the application is closed. This keeps time-critical announcements such as placement drives and last-minute venue changes from being missed. [9]

- Genuine browser push notifications delivered through the Web Push protocol with VAPID keys
- A per-community notification bell lets each user choose which publishers may alert them
- Messages are delivered even when the application is closed or running in the background
- Well suited to time-critical alerts such as placement drives and last-minute venue changes

2.10 Post Scheduling and Automatic Expiry

While composing an announcement a publisher may set an optional “Expires On” date and an optional scheduled publish time. A background job runs every fifteen minutes and moves expired notices into a separate archive table so that feeds stay relevant and uncluttered, while administrators retain a read-only view of archived posts for historical reference. Deletion of posts is restricted to administrators and every deletion is recorded in an audit log.

Conceptual Feature Flow

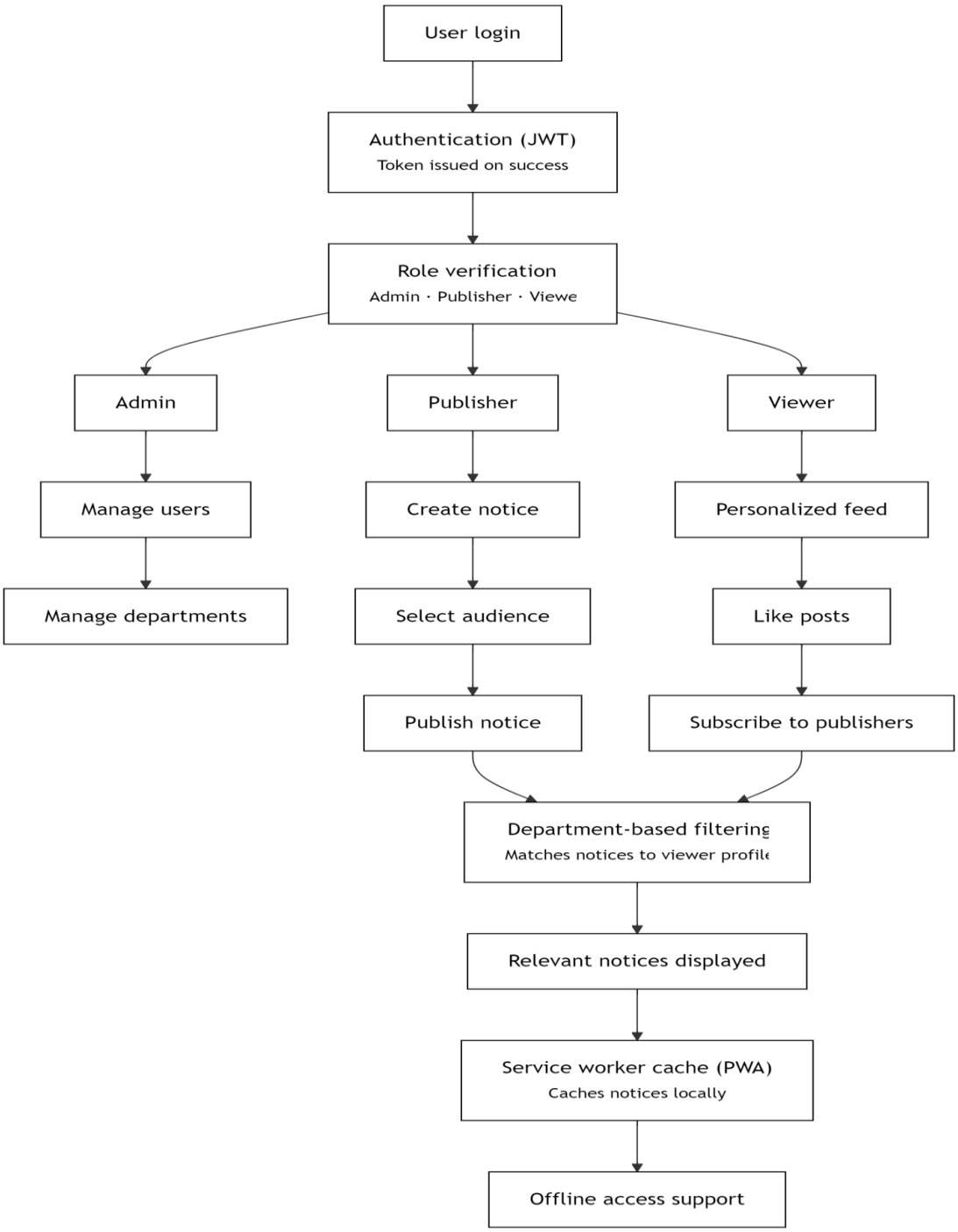

Figure 1: Conceptual Feature Flow of the College Forum App

3. Technology Stack

College Forum is developed using a modern full-stack web development architecture that combines frontend technologies, backend services, database management systems, security mechanisms, and Progressive Web Application (PWA) technologies. The selected technology stack ensures scalability, security, responsiveness, and efficient performance across different devices and platforms.

3.1 Frontend Technologies

The frontend of College Forum is responsible for providing an interactive and user-friendly interface through which users can access the application's features.

HTML5

HTML5 is used to create the structure and layout of the web pages. It provides semantic elements that improve accessibility, maintainability, and compatibility across different browsers. All forms, navigation menus, dashboards, and content sections are built using HTML5.

CSS3

CSS3 is used to design and style the user interface. It enables responsive layouts, animations, color schemes, typography, and visual enhancements that improve the overall user experience.

Bootstrap 5

Bootstrap 5 is utilized as the primary frontend framework for responsive design. It provides pre-built components such as navigation bars, cards, buttons, forms, modals, and grid systems, allowing the application to adapt seamlessly to mobile phones, tablets, laptops, and desktop screens. [7]

JavaScript (Vanilla JavaScript)

Vanilla JavaScript is used to implement client-side functionality without relying on additional frontend frameworks. It handles user interactions, dynamic content rendering, API communication, form validation, authentication token management, and real-time updates within the application.

3.2 Backend Technologies

The backend manages business logic, authentication, authorization, API services, and communication with the database.

Node.js

Node.js serves as the runtime environment for executing server-side JavaScript code. It enables efficient handling of multiple requests through its event-driven and non-blocking architecture, making it suitable for scalable web applications. [1]

Express.js

Express.js is used as the backend web framework. It simplifies routing, middleware integration, request handling, and API development. The framework enables the creation of RESTful APIs that facilitate communication between the frontend and the database. [2]

3.3 Database Technology

MySQL

MySQL is used as the relational database management system for storing and managing application data. It maintains information related to users, departments, posts, subscriptions, likes, and authentication details. MySQL provides high performance, reliability, and structured data management through SQL queries and relational database concepts. [3]

3.4 Authentication and Security Technologies

JSON Web Token (JWT)

JWT is used to implement secure authentication and authorization within the system. After successful login, a token is generated and provided to the user. This token is included in subsequent requests to verify user identity and control access to protected resources. [6]

Password Encryption

User passwords are securely stored in encrypted form using hashing techniques. This prevents unauthorized access to sensitive user credentials and enhances overall system security.

3.5 Progressive Web Application (PWA) Technologies

Manifest.json

The Web App Manifest defines the application's metadata, including its name, icons, theme colors, and installation behavior. It allows users to install College Forum on their devices and access it like a native application. [8]

Service Workers

Service Workers enable offline functionality by caching application resources and previously viewed notices. They intercept network requests and provide cached content when internet connectivity is unavailable. [5]

Cache Storage API

The Cache Storage API is used to store application assets such as HTML pages, CSS files, JavaScript files, images, and API responses. This improves loading speed and supports offline access.

3.6 Development and Deployment Tools

Visual Studio Code

Visual Studio Code is used as the primary Integrated Development Environment (IDE) for writing, debugging, and managing the project source code.

npm (Node Package Manager)

npm is used to manage project dependencies and install required libraries and packages for both frontend and backend development. [10]

Git and GitHub

Git is used for version control, while GitHub is used for source code hosting, collaboration, and project management throughout the development lifecycle.

4. Requirements & Specifications

The successful implementation of College Forum requires a well-defined set of functional and non-functional requirements. These requirements describe the expected behavior, performance, security, and operational characteristics of the system. The specifications ensure that the application meets the communication needs of educational institutions while maintaining reliability, usability, and scalability.

4.1 Functional Requirements

Functional requirements define the core functionalities that the system must provide to its users.

FR1: User Authentication

- The system shall allow registered users to log in using valid credentials.
- The system shall authenticate users using JWT-based authentication.
- The system shall provide role-based access according to user permissions.
- The system shall allow users to securely log out of the application.

FR2: User Management

- The administrator shall be able to create user accounts.
- The administrator shall be able to assign roles such as Admin, Publisher, or Viewer.
- The administrator shall be able to delete user accounts when required.
- The administrator shall be able to assign users to departments.

FR3: Department Management

- The administrator shall be able to create departments.
- The administrator shall be able to view all available departments.
- The administrator shall be able to update department information.
- The administrator shall be able to remove departments when necessary.

FR4: Notice Publishing

- Publishers and administrators shall be able to create notices.
- Publishers shall be able to specify the target audience for each notice.
- Notices can be published to:
    i. All users
    ii. A specific department
    iii. Multiple departments

- Published notices shall be stored in the database.

FR5: Notice Viewing

- Students shall be able to view notices relevant to their department.
- Users shall be able to view institution-wide announcements.
- The system shall display notices in chronological order.

FR6: Post Interaction

- Users shall be able to like posts.
- Users shall be able to remove previously added likes.
- Each user shall be allowed only one active like per post.

FR7: Publisher Subscription

- Users shall be able to subscribe to publishers.
- Users shall be able to unsubscribe from publishers.
- Users shall be able to view their subscribed publishers.

FR8: Offline Access

- The application shall cache recent notices.
- Users shall be able to access previously loaded content without an internet connection.
- The system shall provide an offline fallback page when connectivity is unavailable.

FR9: Progressive Web Application Features

- Users shall be able to install the application on supported devices.
- The application shall support mobile and desktop environments.
- The application shall provide an app-like user experience.

4.2 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system.

Performance Requirements

- The system shall load pages within a reasonable response time.
- API responses shall be processed efficiently.
- The application shall support multiple concurrent users without significant performance degradation.

Reliability Requirements

- The system shall ensure continuous availability of stored notices.
- Offline functionality shall provide access to cached information.
- Data consistency shall be maintained across all modules.

Security Requirements

- User authentication shall be protected using JWT tokens.
- Passwords shall be stored in encrypted form.
- Unauthorized users shall not access protected resources.
- Role-based authorization shall restrict access to sensitive operations.

Scalability Requirements

- The architecture shall support the addition of new users and departments.
- New modules and features should be integrated without major architectural changes.
- The database design shall support future growth.

Maintainability Requirements

- The system shall follow a modular architecture.
- Source code shall be easy to understand and maintain.
- Components shall be reusable and independently manageable.

Usability Requirements

- The user interface shall be intuitive and easy to navigate.
- The application shall provide a responsive experience across different devices.
- Users should be able to perform common tasks with minimal training.

Compatibility Requirements

- The application shall function on modern web browsers.
- The system shall support desktop, tablet, and mobile devices.
- The PWA shall operate consistently across supported platforms.

4.5 System Specifications

Table 1: System Specifications

Category	Specification
Application Type	Progressive Web Application (PWA)
Architecture	Client-Server Architecture
Frontend	HTML5, CSS3, Bootstrap 5, JavaScript
Backend	Node.js, Express.js
Database	MySQL

Category	Specification
Authentication	JSON Web Token (JWT)
User Roles	Admin, Publisher, Viewer
Deployment Type	Web-Based Application
Offline Support	Service Workers and Cache Storage
Device Support	Mobile, Tablet, Desktop

Table 1 presents the complete technical configuration on which the College Forum App is built.

the application is implemented as a Progressive Web Application following a client-server architecture, using HTML5, CSS3, Bootstrap 5, and JavaScript on the frontend, Node.js and Express.js on the backend, and MySQL for data storage. This stack underpins the offline access, installability, push notifications, and responsive behaviour described throughout the report.

5. User Interface Mockups / Wireframes

The user interface of College Forum is designed using a mobile-first approach to ensure accessibility across smartphones, tablets, laptops, and desktop devices. The interface focuses on simplicity, ease of navigation, and efficient access to important information. The following wireframes provide a conceptual representation of the major screens within the application.

5.1 Login Screen

The Login Screen serves as the entry point to the application. Users must provide valid credentials to access their respective dashboards. JWT authentication is used to verify user identity and establish a secure session. The login interface is shown in Figure 2.

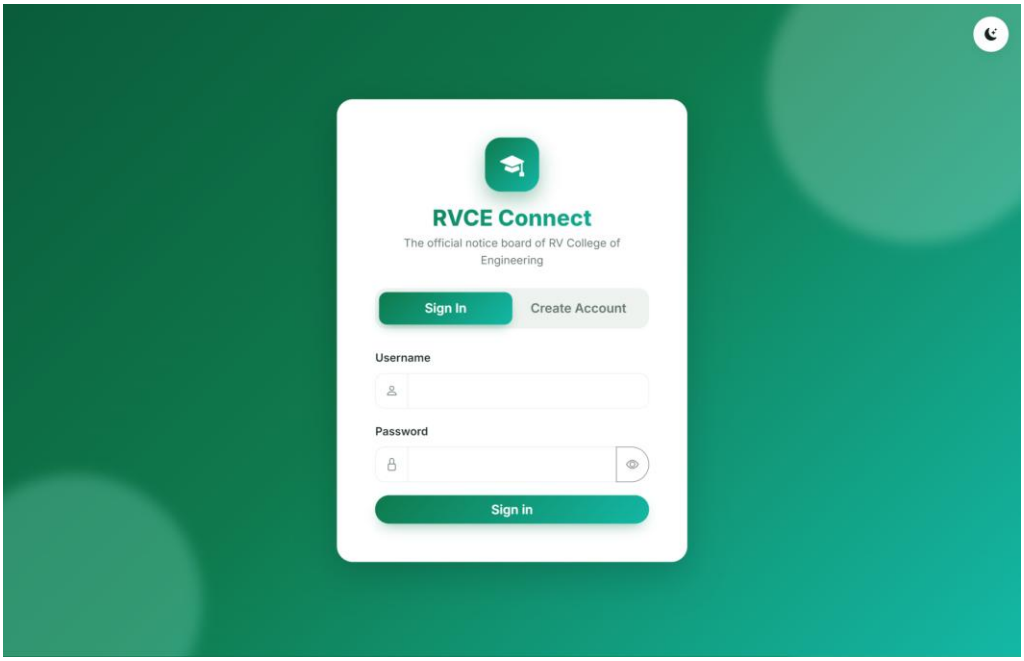

Figure 2: Login screen of the College Forum App

Purpose

- Authenticate users securely.
- Redirect users based on assigned roles.
- Provide access to protected resources.

5.2 Student (Viewer) Dashboard

The Viewer Dashboard displays a personalized feed containing department-specific and institution-wide announcements. This dashboard is shown in Figure 3.

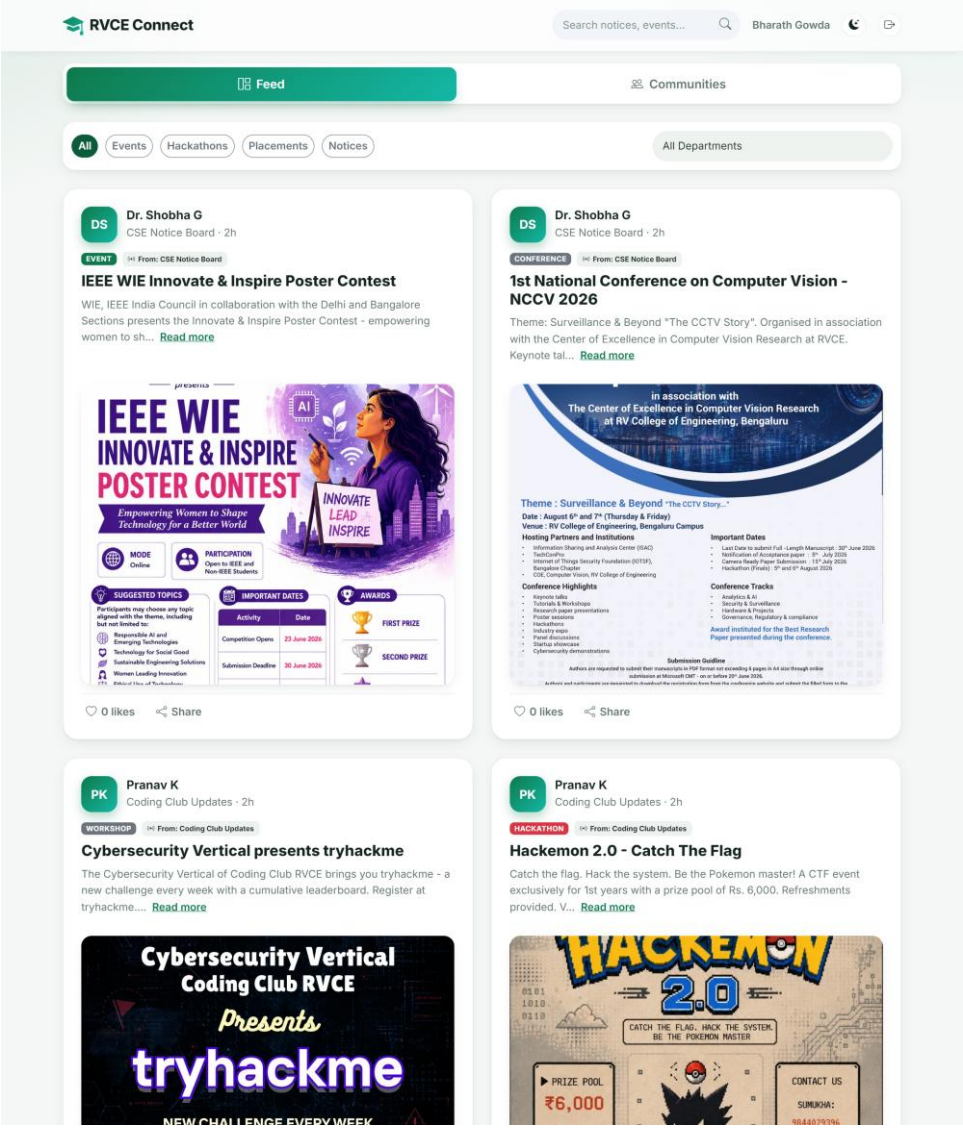

Figure 3: Student (viewer) feed dashboard

Purpose

- Display personalized notices.
- Allow users to like posts.
- Enable subscriptions to publishers.

5.3 Publisher Dashboard

Publishers use this interface to create and manage announcements. The publisher interface is shown in Figure 4.

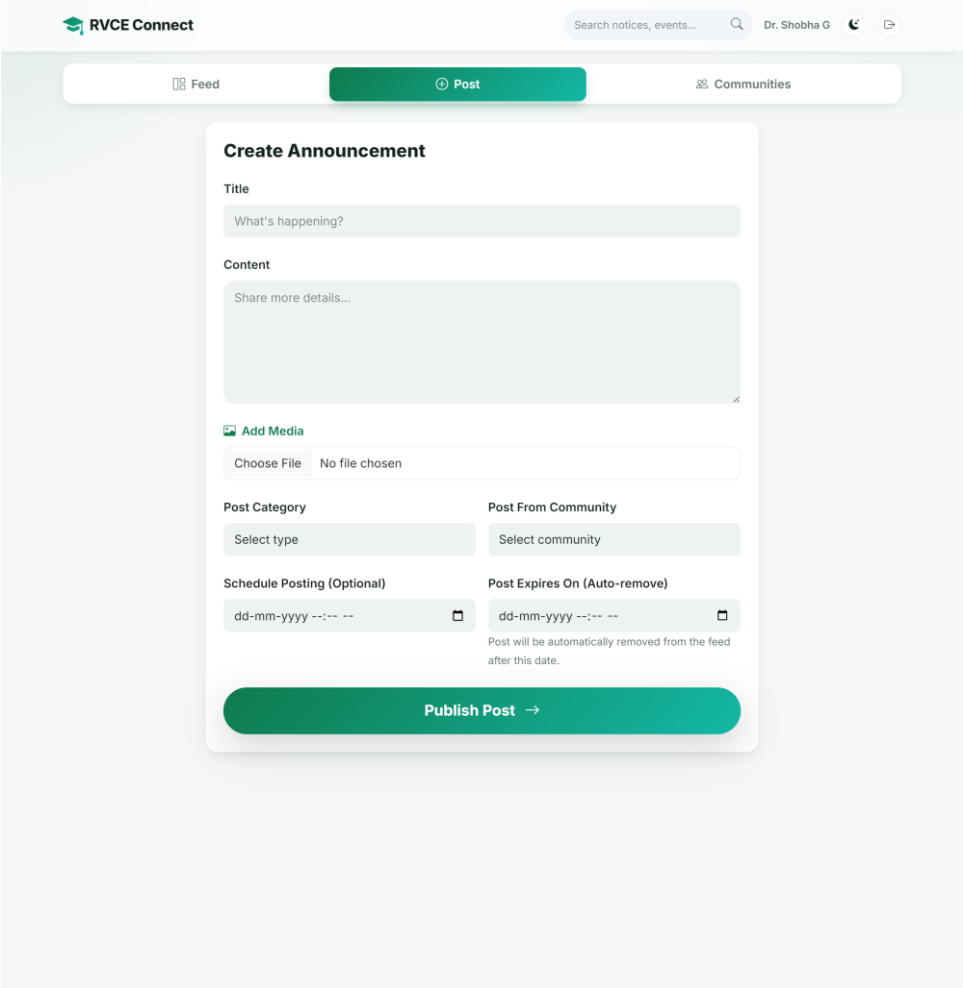

Figure 4: Publisher create-announcement screen

- Create announcements.
- Select target audience.
- Publish department-specific notices.

5.4 Admin Dashboard

The Admin Dashboard provides complete control over system management. The admin dashboard is shown in Figure 5.

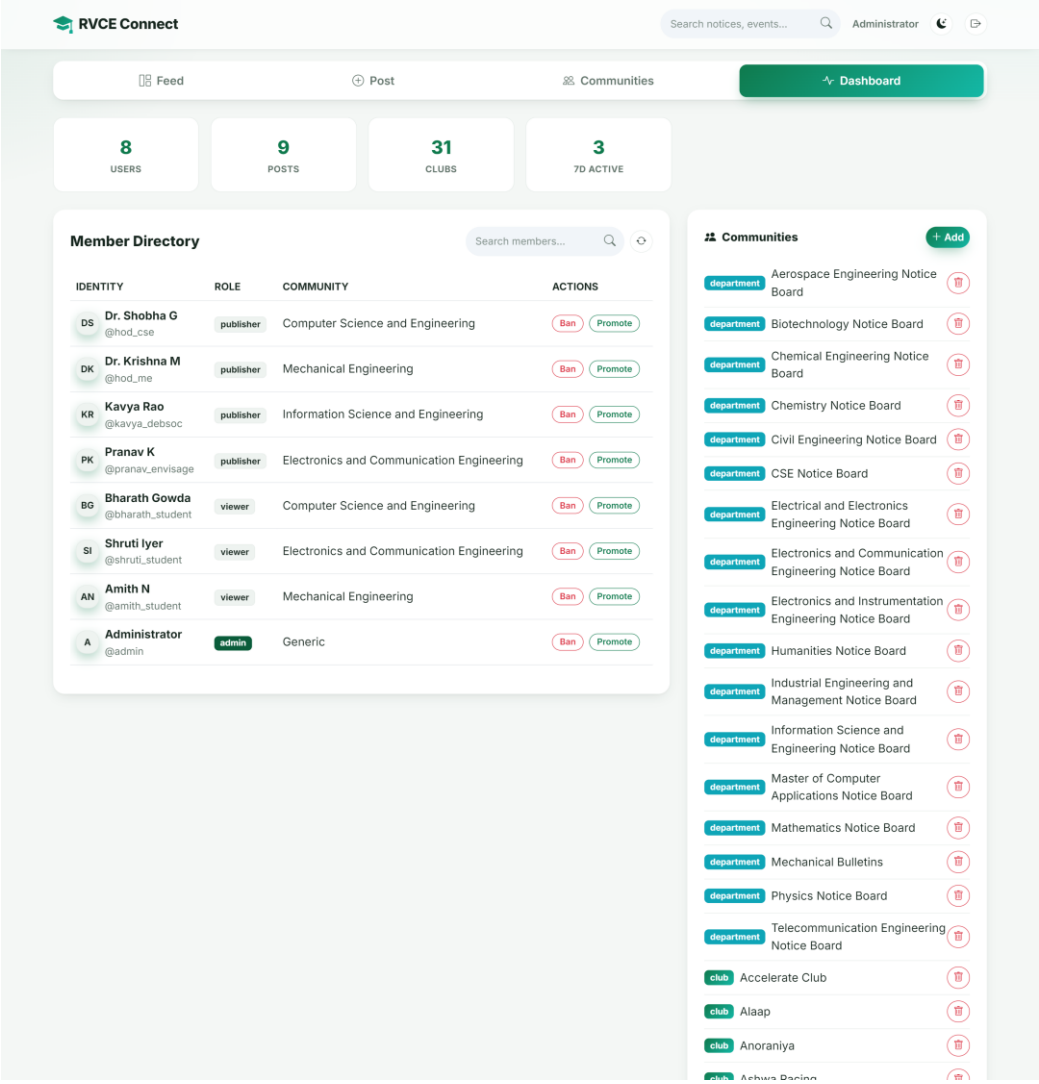

Figure 5: Admin dashboard – member directory and community management

Purpose

- Manage users.
- Assign roles.
- Control departments.
- Monitor platform activities.

5.5 Department Management Screen

Administrators can create and manage departments from this interface. This screen is shown in Figure 6.

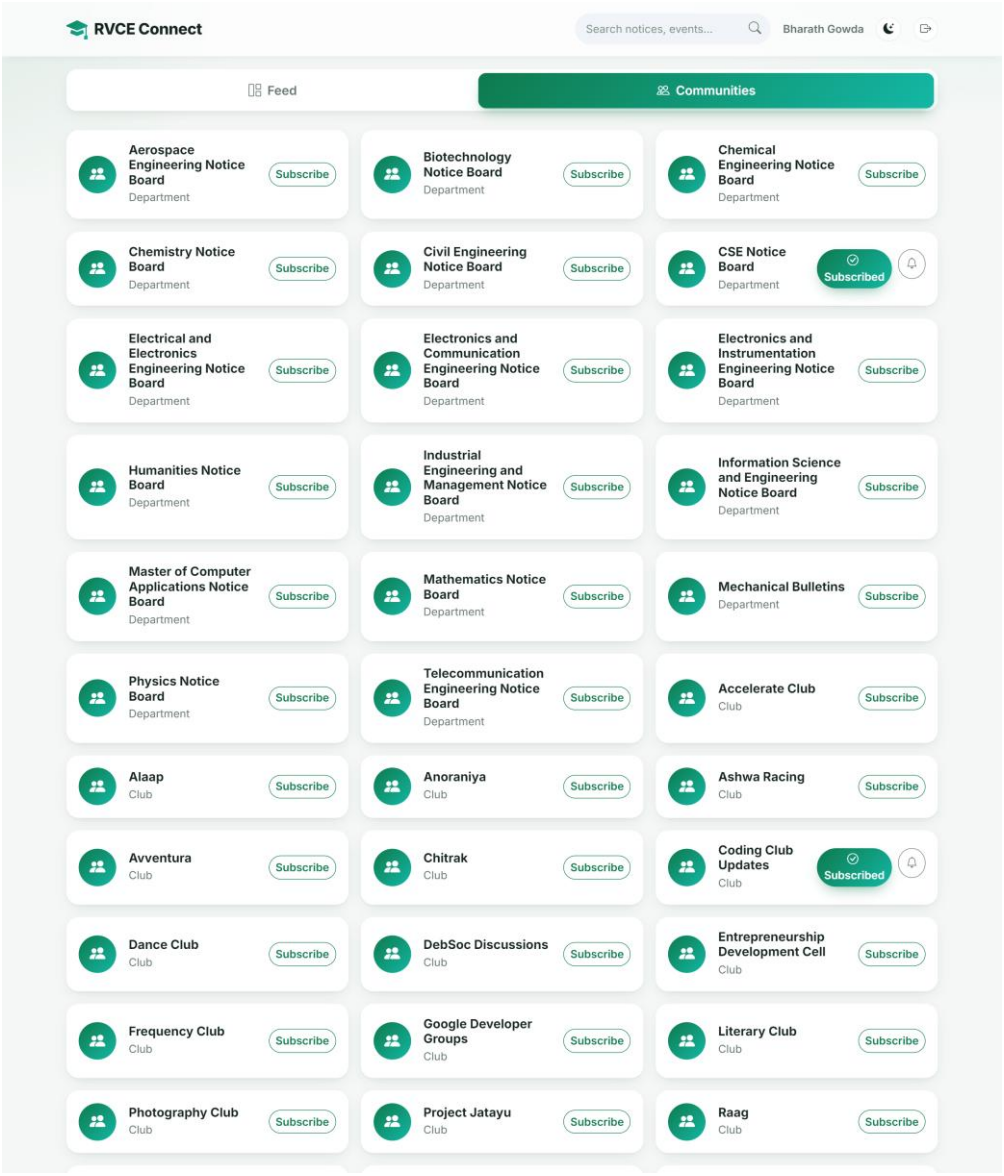

Figure 6: Communities directory (departments and clubs)

Purpose

- Add new departments.
- View available departments.
- Organize user assignments.

5.6 Publisher Subscription Screen

Users can manage subscriptions to departments, clubs, and faculty publishers. The subscription screen is shown in Figure 7.

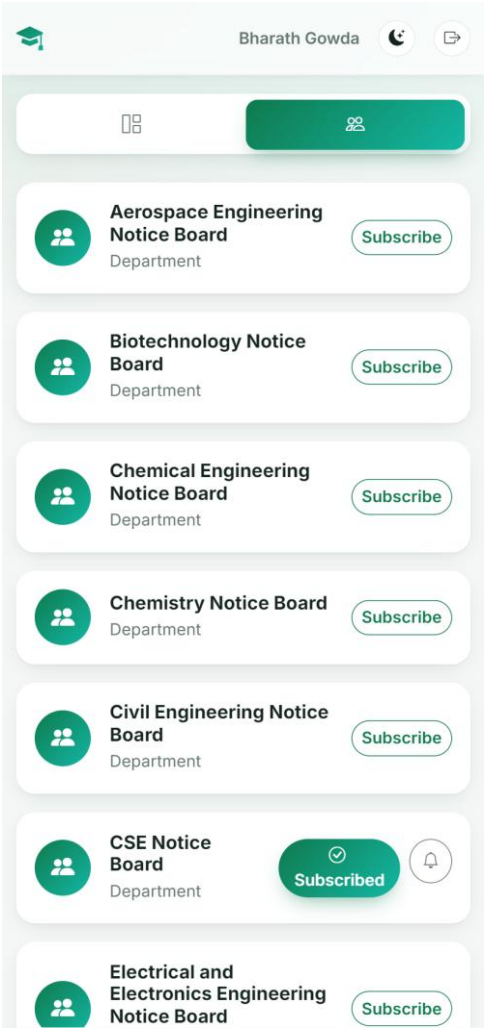

Figure 7: Subscribing to a community

Purpose

- Follow preferred publishers.
- Receive personalized updates.
- Improve user engagement.

5.7 Offline Page

This screen appears when the application cannot establish an internet connection and no recent data is available in the cache. The offline fallback page is shown in Figure 8.

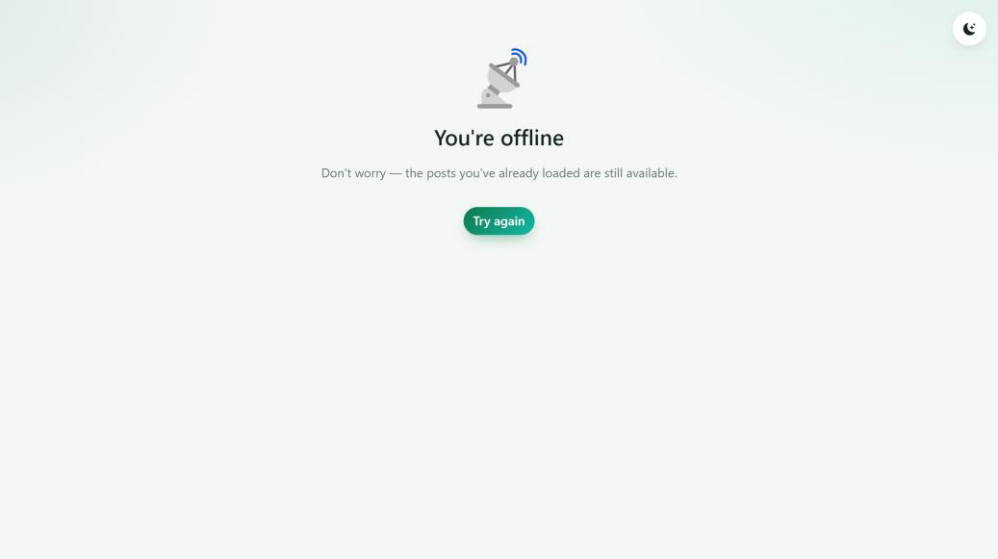

Figure 8: Offline fallback page

Purpose

- Inform users about connectivity issues.
- Provide access to cached content.
- Enhance application reliability.

5.8 Mobile Navigation Layout

Since most users access the platform through smartphones, a bottom navigation bar is provided for quick access. The mobile navigation layout is shown in Figure 9.

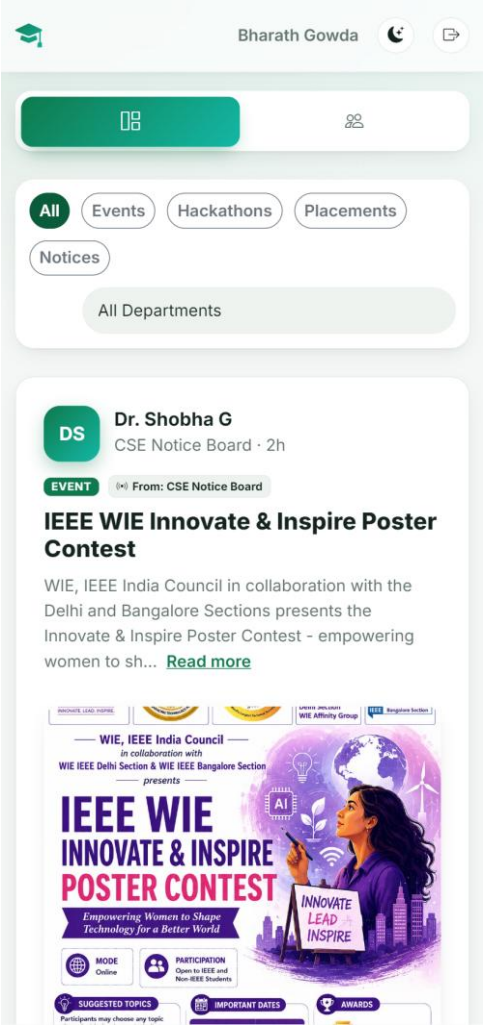

Figure 9: Mobile-first responsive layout

- Simplify navigation.
- Improve accessibility.
- Provide a mobile-friendly experience.

5.9 User Interface Design Analysis

The user interface design follows the principles of simplicity, consistency, responsiveness, and accessibility. Key design considerations include:

- Mobile-first responsive design using Bootstrap 5.
- Clean and minimalistic layouts for better readability.
- Consistent navigation across all screens.
- Role-specific dashboards for improved usability.
- Intuitive controls for publishing, subscribing, and interacting with notices.

- Offline support indicators to enhance user experience.

The proposed wireframes provide a clear visual representation of how users interact with College Forum and demonstrate the application's focus on efficient communication, ease of use, and accessibility across multiple devices.

5.10. Navigation Flow

The navigation flow of College Forum defines how users move through the application and interact with its various features. The system follows a role-based navigation approach, ensuring that each user is presented with functionalities relevant to their assigned role. The navigation structure is designed to be simple, intuitive, and responsive, allowing users to access information and perform tasks efficiently.

When a user opens the application, they are directed to the Login Screen, where they must enter valid credentials. Upon successful authentication, the system verifies the user's role using JWT-based authorization and redirects them to the appropriate dashboard. This role-based routing ensures secure access control and prevents unauthorized users from accessing restricted functionalities.

For viewers (students), the primary navigation revolves around accessing and interacting with notices. After login, viewers are directed to the Feed Dashboard, where they can view department-specific and institution-wide announcements. From the feed, users can navigate to subscriptions to follow publishers, access their profile information, and interact with notices through likes. The navigation is designed to provide quick access to relevant information while maintaining a clutter-free interface.

Publishers have additional navigation options that allow them to create and manage announcements. After logging in, publishers can access the Create Post module, where they can compose notices and select the intended audience. They can also navigate to the My Posts section to review or delete previously published announcements. This workflow enables efficient content management while ensuring that notices reach the correct audience.

Administrators have the highest level of access within the system. Their navigation includes modules for user management, department management, and system monitoring. Administrators can create departments, add or remove users, assign roles, and oversee platform activities. These administrative functions are accessible through a dedicated dashboard designed specifically for system management and control.

The application also integrates Progressive Web Application (PWA) navigation features. Users can install the application on their devices and access it through a home screen icon, similar to a native mobile application. When internet connectivity is unavailable, cached pages and previously viewed notices remain accessible through offline navigation supported by Service Workers. This ensures uninterrupted access to important information and enhances the overall user experience. The complete navigation flow is illustrated in Figure 10.

Navigation Flow Diagram

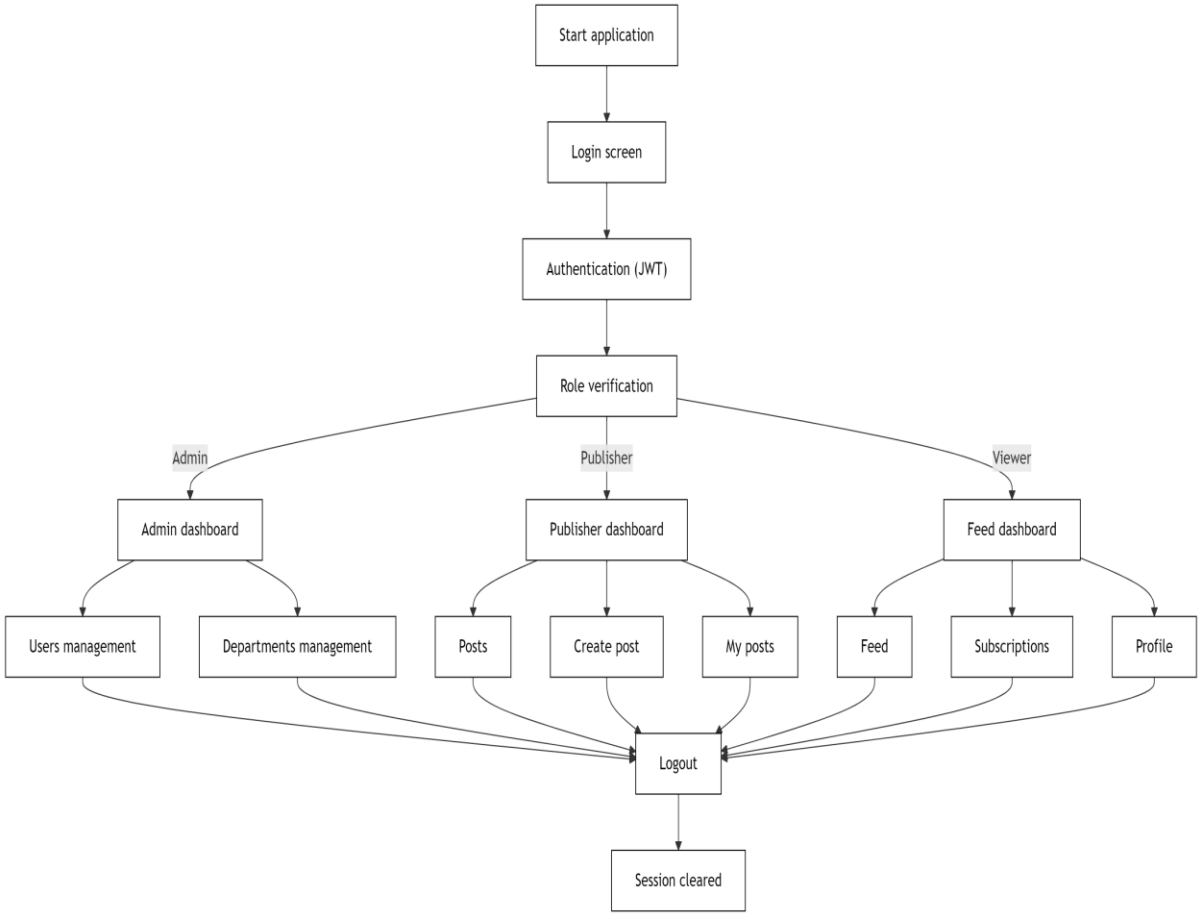

Figure 10: Navigation Flow of the College Forum App

6. Implementation Details

The implementation of College Forum follows a full-stack web development architecture consisting of a frontend layer, backend layer, database layer, authentication system, and Progressive Web Application (PWA) components. The system is developed using HTML5, CSS3, Bootstrap 5, JavaScript, Node.js, Express.js, and MySQL. The architecture is designed to be modular, scalable, and maintainable, allowing each component to operate independently while working together as a unified system.

The frontend is responsible for providing an interactive user interface through which users can access application functionalities. It is developed using HTML5, CSS3, Bootstrap 5, and Vanilla JavaScript. The frontend communicates with the backend through RESTful API calls and dynamically updates content based on user actions. Responsive design principles are applied to ensure compatibility across mobile devices, tablets, and desktop systems.

The backend is developed using Node.js and Express.js and serves as the central processing unit of the application. It handles user authentication, authorization, business logic, API

requests, and communication with the database. Express routing is used to organize application functionalities into separate modules, improving code maintainability and readability. The backend validates user requests, enforces role-based permissions, and manages data transactions securely.

MySQL is used as the relational database management system for storing user accounts, departments, posts, likes, and subscriptions. Database relationships are established through primary and foreign keys to maintain data consistency and integrity. SQL queries are executed through the backend to retrieve, insert, update, and delete information as required by the application. [3]

The system implements JWT (JSON Web Token) authentication to secure access to protected resources. Upon successful login, a token is generated and provided to the user. This token accompanies future API requests and is verified before granting access to system functionalities. This approach ensures secure and stateless authentication throughout the application.

As a Progressive Web Application, College Forum utilizes Service Workers and Cache Storage APIs to support offline access. Application resources and recently viewed notices are cached locally, allowing users to access important information even when internet connectivity is unavailable. This enhances reliability and improves the overall user experience.

Core Logic and Module Descriptions

The functionality of College Forum is divided into several modules, each responsible for a specific aspect of the system. These modules work together to provide secure communication, efficient notice management, and seamless user interaction.

6.1 Authentication Module

Purpose

The Authentication Module verifies user credentials and manages secure access to the application.

Working

1. User enters username and password.
2. Credentials are sent to the backend.
3. The system verifies user information stored in the database.
4. Upon successful verification, a JWT token is generated.
5. The token is returned to the client and stored locally.
6. Subsequent API requests include the token for authentication.

Outcome

- Secure login process.
- Protected access to application resources.

- Role-based authorization support.

6.2 User Management Module

Purpose

Allows administrators to manage users and assign roles.

Functionalities

- Create user accounts
- Delete users
- Assign departments
- Assign roles

Outcome

Ensures proper organization and control of system users.

6.3 Department Management Module

Purpose

Maintains academic departments within the institution.

Functionalities

- Create departments
- View department list
- Update department information
- Remove departments

Outcome

Provides a structured environment for department-based communication.

6.4 Notice Management Module

Purpose

Handles the creation, storage, and display of announcements.

Working

1. Publisher creates a notice.
2. Audience is selected.
3. Notice is validated.
4. Notice is stored in the database.
5. Relevant users receive the notice in their feed.

Outcome

Efficient dissemination of information to targeted users.

6.5 Feed Generation Module

Purpose

Provides personalized notice feeds for users.

Working

The system filters notices based on:

- User department
- Audience selection
- Institution-wide visibility

Outcome

Users receive only relevant announcements.

6.6 Like Management Module

Purpose

Allows users to interact with notices through likes.

Working

1. User clicks Like.
2. System checks if the user already liked the post.
3. If not liked, a new record is inserted.
4. If already liked, the like is removed.

Outcome

Improves engagement and measures notice popularity.

6.7 Subscription Module

Purpose

Allows users to follow publishers such as departments and clubs.

Functionalities

- Subscribe
- Unsubscribe
- View subscriptions

Working

Subscriptions are stored in the database and linked to user accounts.

Outcome

Provides a personalized communication experience.

6.8 Authorization Module

Purpose

Controls access to application functionalities based on user roles.

Access Rules

Table 2: Role-Based Access Rules

Role	Permissions
Admin	Full Access
Publisher	Create and Manage Notices
Viewer	View and Interact with Notices

Table 2 defines the permission boundary for each role: administrators have full control over system management, publishers are limited to creating and managing notices, and viewers may only read and interact with notices. These rules are enforced by the Authorization Module to guarantee controlled and secure access.

Outcome

Ensures system security and controlled access.

6.9 Offline Access Module

Purpose

Provides access to content without internet connectivity.

Working

1. Service Worker caches application resources.
2. Recent notices are stored locally.
3. Cached content is served when offline.

Outcome

Ensures uninterrupted access to important notices.

6.10 API Communication Module

Purpose

Facilitates communication between frontend and backend components.

Major APIs

Table 3: Major API Endpoints

API Endpoint	Purpose
/api/auth/login	User Authentication
/api/auth/me	Current User Details
/api/posts	Retrieve Feed
/api/posts/:id/like	Like Notice
/api/users	Manage Users
/api/departments	Manage Departments

API Endpoint	Purpose
/api/subscriptions	Manage Subscriptions

Table 3 lists the principal REST API endpoints exposed by the backend together with the purpose of each. Collectively these endpoints handle authentication, feed retrieval, post interaction, and user management, and form the communication contract between the frontend Progressive Web Application and the server.

Outcome

Enables seamless data exchange between client and server.

6.11 Overall System Workflow

The implementation of these modules ensures that College Forum functions as a secure, scalable, and efficient communication platform. The modular architecture simplifies maintenance, enhances system reliability, and supports future feature enhancements while providing a seamless user experience for administrators, publishers, and students. The overall workflow is illustrated in Figure 11.

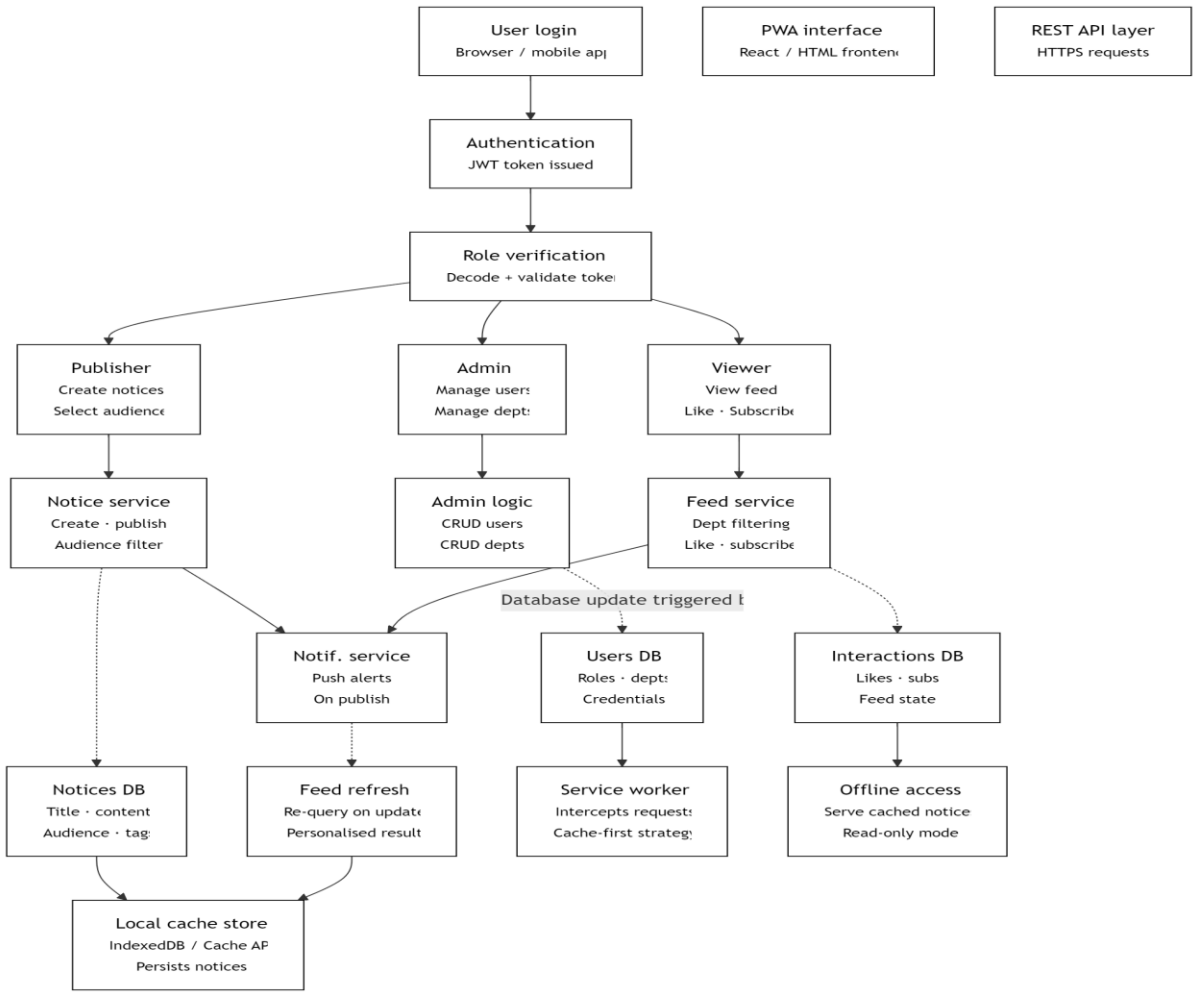

Figure 11: Overall System Workflow Architecture

7. Demonstration

The demonstration of College Forum showcases the key functionalities of the application and illustrates how different users interact with the system based on their assigned roles. The demonstration focuses on user authentication, notice publishing, department-specific content delivery, user interactions, subscription management, and offline accessibility. Through this demonstration, the effectiveness of the platform in improving communication within educational institutions can be clearly observed.

The demonstration begins with the login process, where users enter their credentials to access the application. After successful authentication, the system verifies the user's role and redirects them to the appropriate dashboard. Administrators are provided with management functionalities, publishers can create and manage notices, and viewers can access personalized announcements and interact with available content.

The administrator demonstration highlights user and department management features. The administrator can create departments, add new users, assign roles, and manage the overall structure of the system. These operations ensure that the platform remains organized and that only authorized individuals have access to publishing privileges.

The publisher demonstration focuses on content creation and distribution. Publishers can create announcements, specify the target audience, and publish notices to selected departments or to all users. Once published, the notices become immediately available in the feeds of relevant users. This demonstrates the system's ability to deliver targeted information efficiently.

The viewer demonstration showcases how students access and interact with information. Students can browse department-specific notices, view institution-wide announcements, like posts, and subscribe to publishers such as departments, faculty members, or student organizations. This personalized experience ensures that users receive updates that are relevant to their interests and academic activities.

Finally, the offline functionality of the Progressive Web Application is demonstrated by accessing previously loaded notices without an internet connection. Through Service Workers and caching mechanisms, the application continues to provide access to important information, ensuring reliability even in environments with limited connectivity.

7.1 Demonstration Description

Step 1: User Authentication

1. Open the College Forum application.
2. Enter valid username and password.
3. Click the Login button.

4. The system authenticates the user and redirects them to the appropriate dashboard.

Expected Result: Secure login and role-based dashboard access.

Step 2: Administrator Operations

1. Login as an Administrator.
2. Navigate to the User Management section.
3. Create a new user account.
4. Assign a role and department.
5. Navigate to Department Management.
6. Add a new department.

Expected Result: New users and departments are successfully created and stored in the database.

Step 3: Publisher Operations

1. Login as a Publisher.
2. Open the Create Post page.
3. Enter the notice title and content.
4. Select the target audience (Everyone or specific departments).
5. Click Publish.

Expected Result: Notice is successfully published and becomes visible to the selected audience.

Step 4: Viewer Operations

1. Login as a Viewer (Student).
2. Open the Feed page.
3. View department-specific and institution-wide notices.
4. Like a notice.
5. Subscribe to a publisher.

Expected Result: Feed displays relevant notices, likes are recorded, and subscriptions are successfully added.

Step 5: Offline Access Demonstration

1. Open the application while connected to the internet.
2. Browse several notices.
3. Disconnect the internet connection.
4. Refresh the application.
5. Access previously viewed notices.

Expected Result: Cached notices remain accessible through the Service Worker.

Figures 12 and 13 present representative screenshots captured during the live demonstration, showing new-student self-registration and the campus feed immediately after a publisher posts an announcement.

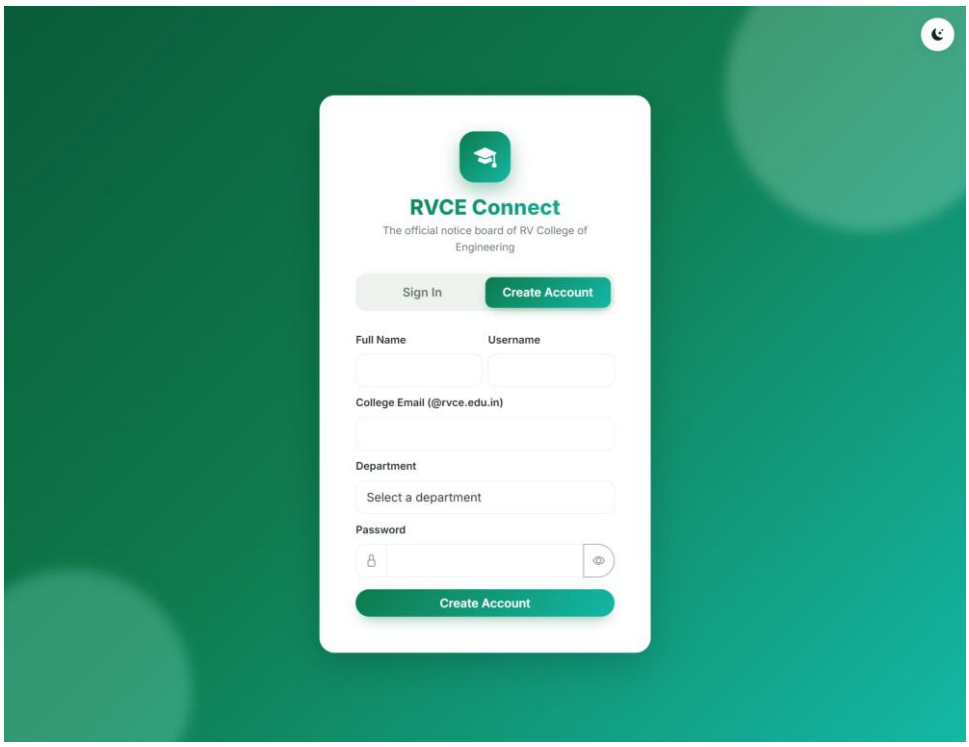

Figure 12: New-student self-registration

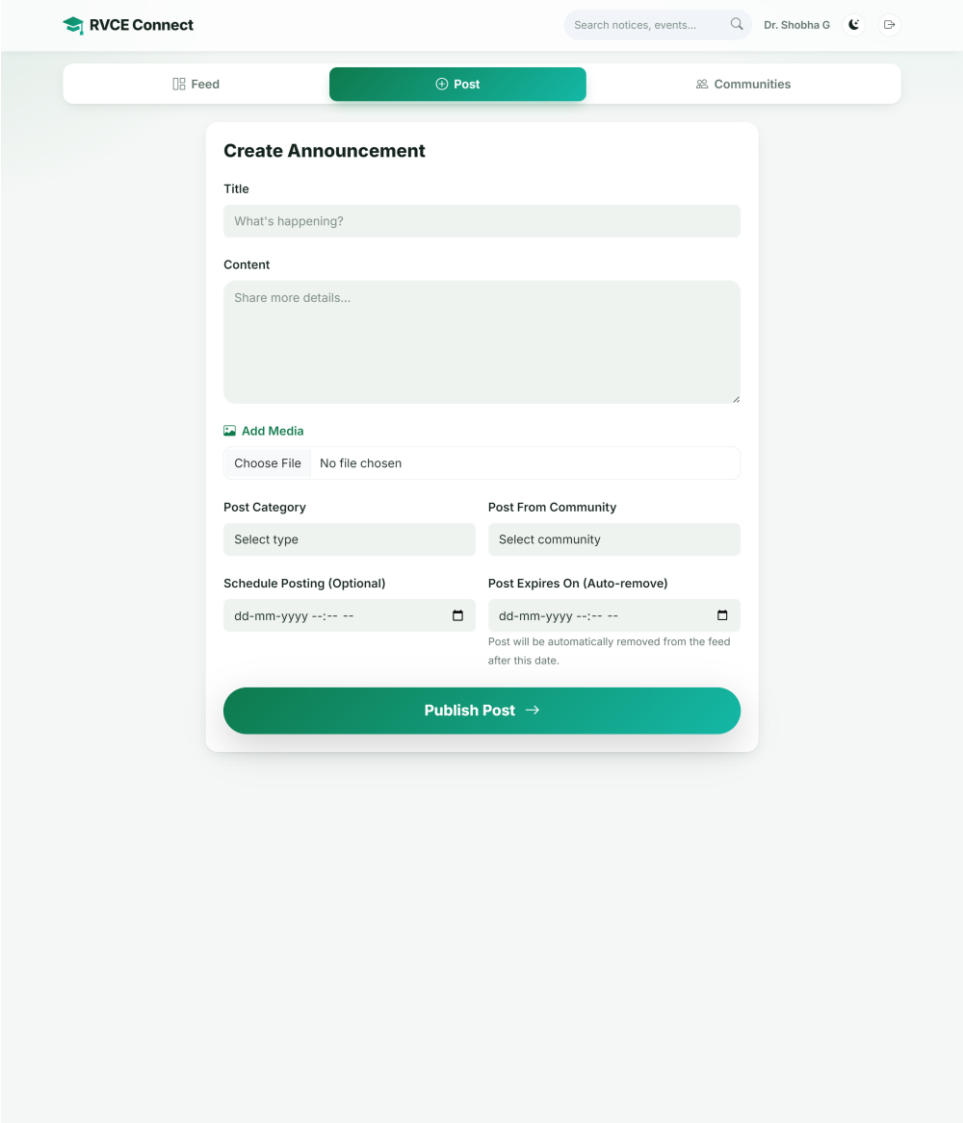

Figure 13: Campus feed after a publisher posts an announcement

7.2 Demonstration Flow

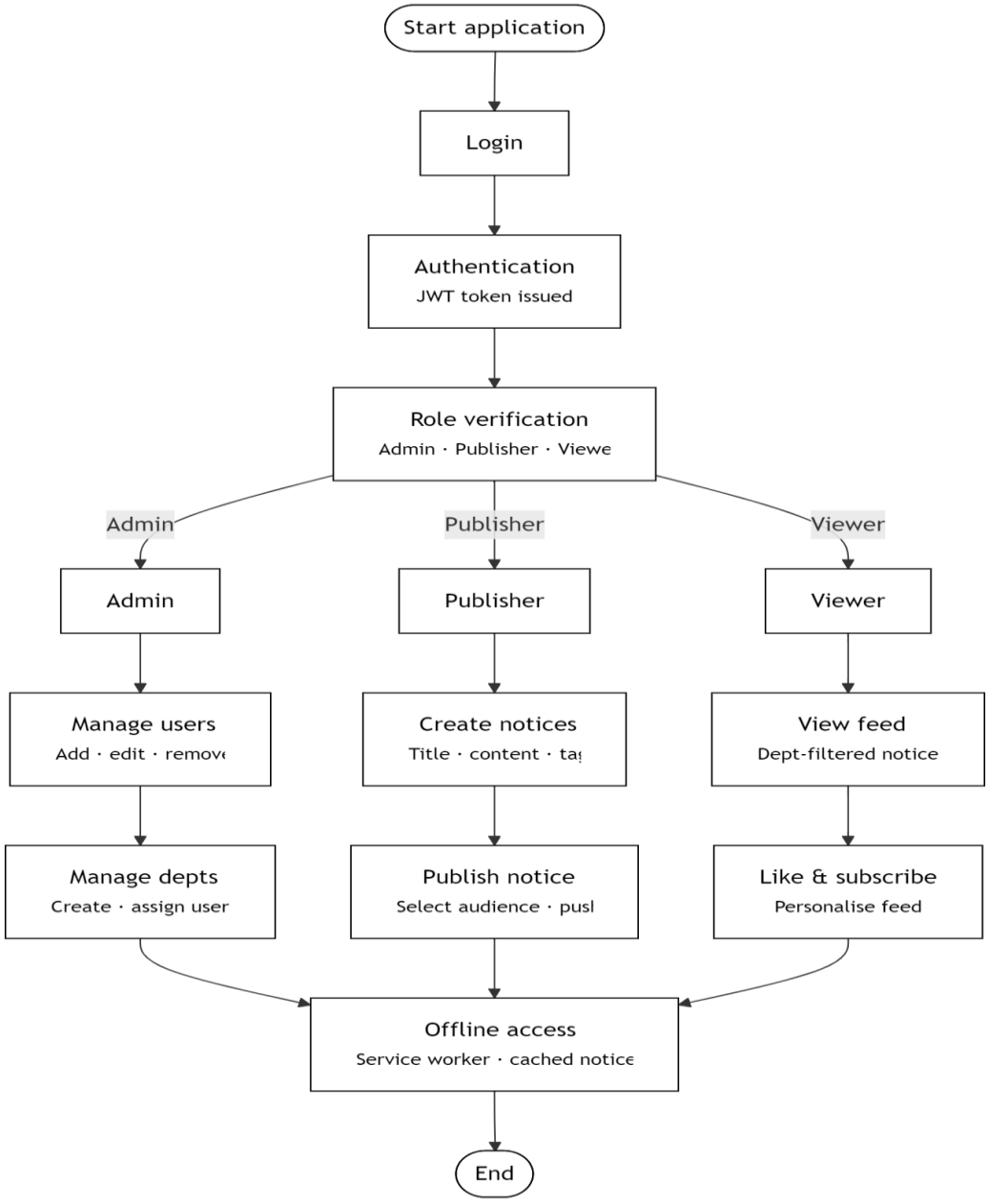

Figure 14: Demonstration Flow of the College Forum App

7.3 Demonstration Link or Description

Demonstration Link

Project URL: http://localhost:3000 *(Development Environment)*

GitHub Repository:https://github.com/Sumukha-Rao/campus-connect

Demonstration Environment

Table 4 lists the software environment in which the application was demonstrated.

Table 4: Demonstration Environment Specifications

Component	Configuration
Frontend	HTML5, CSS3, Bootstrap 5, JavaScript
Backend	Node.js, Express.js
Database	MySQL 8
Authentication	JWT
Browser	Google Chrome / Microsoft Edge
Platform	Progressive Web Application (PWA)

Table 4 records the exact software environment used for the demonstration, documenting the frontend, backend, database, authentication mechanism, and browser configuration so that the demonstration can be reliably reproduced.

8. Conclusion

College Forum was developed to address the communication challenges commonly faced by educational institutions, where important information is often scattered across physical notice boards, emails, and multiple messaging platforms. The project successfully provides a centralized digital communication platform that enables efficient distribution of academic, administrative, and extracurricular announcements. By integrating role-based access control, department-specific notice delivery, publisher subscriptions, and interactive engagement features, the system ensures that relevant information reaches the appropriate audience in a timely and organized manner.

The implementation of College Forum as a Progressive Web Application further enhances accessibility and usability by providing a responsive, mobile-friendly, and installable platform. Features such as offline access through Service Workers and cached content improve reliability and ensure continuous availability of information even in situations with limited internet connectivity. The use of modern web technologies, secure authentication mechanisms, and a scalable architecture makes the system suitable for real-world deployment within colleges, universities, and other educational institutions.

The project demonstrates how digital transformation can significantly improve information management and communication efficiency in academic environments. By replacing

traditional notice boards with a centralized, interactive platform, College Forum promotes greater engagement among students, faculty, departments, and administrators. The modular architecture also enables future enhancements, including push notifications, event management, analytics dashboards, profile management, and mobile application integration.

Overall, College Forum successfully achieves its objective of creating a secure, efficient, and user-friendly campus communication system. The project not only solves a practical problem within educational institutions but also showcases the effective application of full-stack web development concepts and Progressive Web Application technologies in building scalable real-world solutions.

8.1 Contributions of the Project

The major contributions of the College Forum project are as follows:

- Developed a centralized digital notice board system for educational institutions.
- Implemented role-based access control with Admin, Publisher, and Viewer roles.
- Enabled department-specific announcement targeting to improve communication efficiency.
- Designed a personalized notice feed for students and other users.
- Implemented post interaction features such as likes and publisher subscriptions.
- Developed secure user authentication and authorization using JSON Web Tokens (JWT).
- Created a responsive user interface using Bootstrap 5 and mobile-first design principles.
- Integrated Progressive Web Application (PWA) features including installation support and offline accessibility.
- Designed and implemented a relational database using MySQL for efficient data management.
- Built RESTful APIs using Node.js and Express.js to facilitate communication between frontend and backend components.

8.2 Academic Learning Outcomes

The project provided practical exposure to several concepts studied in software engineering and web development courses, including:

- Client-Server Architecture.
- Database Design and Management.

- Authentication and Authorization Techniques.
- Progressive Web Application Development.
- User Interface and User Experience Design.
- Software Requirement Analysis.
- System Design and Implementation.
- Testing and Deployment Concepts.

8.3 Future Scope

The project can be extended further by incorporating advanced features such as:

- Event registration and attendance management.
- Notice categorization and advanced search functionality.
- User profile management and password recovery.
- Analytics dashboards for administrators and publishers.
- Mobile application development using cross-platform technologies.
- Integration with institutional ERP and Learning Management Systems (LMS).

These enhancements would further improve the functionality and effectiveness of the platform while increasing its applicability across a wider range of educational environments.

9. References

[1] OpenJS Foundation, “Node.js Documentation.” [Online]. Available: https://nodejs.org/en/docs

[2] OpenJS Foundation, “Express – Fast, unopinionated, minimalist web framework for Node.js.” [Online]. Available: https://expressjs.com

[3] Oracle Corporation, “MySQL 8.0 Reference Manual.” [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[4] Mozilla, “Progressive web apps (PWAs),” MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps

[5] Mozilla, “Service Worker API,” MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API

[6] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519

[7] Bootstrap Team, “Bootstrap 5 Documentation.” [Online]. Available: https://getbootstrap.com/docs/5.3/

[8] W3C, “Web Application Manifest.” [Online]. Available: https://www.w3.org/TR/appmanifest/

[9] M. Thomson, “Voluntary Application Server Identification (VAPID) for Web Push,” RFC 8292, IETF, Nov. 2017. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8292

[10] npm, Inc., “npm Documentation.” [Online]. Available: https://docs.npmjs.com

[11] J. Wilander et al., “bcrypt.js – Optimized bcrypt in JavaScript.” [Online]. Available: https://github.com/dcodeIO/bcrypt.js